

Project Spec: Model Evaluation Harness

What you are building

A command-line tool that runs a set of language models against a fixed set of test questions, scores their answers, and produces a report comparing them. By the end you should be able to type one command, walk away, and come back to a table that says how each model did and how much it cost.

Once you have something working, we will run some models through it together and see where it goes from there.

Why this project

You will touch most of the things a working engineer deals with: API authentication, JSON, loops, error handling, concurrency, data storage, and basic statistics. None of it is hard in isolation. Putting it together and making it reliable is the skill.

There is also a second lesson that matters more than the code: most benchmark numbers are less trustworthy than they look. By the end you should be able to explain why "Model A scored 87%" is often close to meaningless without more context. If you finish the code but cannot explain that, you have missed the point of the project.

A note on using AI

You already know that an AI model could write this entire project in a few minutes, probably better than either of us would on a first pass. So why are you doing it by hand? Because the finished code was never the point. The point is the wiring that forms in your head while you build it: why an API call fails, what a rate limit feels like, why your score moved when you changed nothing. You cannot get that secondhand. If you have the model write it, you end up with a working tool and an empty head, and the first time it breaks in a way the model did not anticipate, you are stuck with no idea what you are looking at.

There is a sharper irony here too. This project is about learning when to trust a model's output and when not to. Handing the build to a model to avoid the work is the exact instinct the project is meant to train you out of.

So here is the line. Use AI the way you would use a tutor or a reference, not the way you would use a vending machine. Asking it to explain a concept you do not understand, or to help you read an error message, is fine and encouraged. Having it generate code you then paste in without understanding is not, and it will be obvious in the walkthrough at the end when you cannot explain your own project. When in doubt: if you used help to produce a piece of code, you should be able to delete it and rewrite it yourself from understanding. If you cannot, you do not understand it yet.

Ground rules

- Commit to git from day one. Small, frequent commits with real messages.
- Every phase below has acceptance criteria. Do not move to the next phase until the current one meets them. Show me before moving on.
- When something breaks, read the actual error message before doing anything else. Most of your time will be spent here, and that is normal.
- Do not paste in code you cannot explain line by line. If you used help to write it, understand it before committing it.
- Keep a running notes file (`NOTES.md`) of decisions, things that broke, and questions. Bring it to check-ins.

Tech stack

Use these defaults unless you have a specific reason not to, and if you deviate, write down why in `NOTES.md`.

- Language: Python 3.11 or newer
- HTTP: start with the `requests` library, or the official SDK of whichever model provider we use
- Config: a plain JSON or YAML file, not hardcoded values
- Storage: write results to JSON files first. Move to SQLite only if and when JSON gets painful.
- Analysis: `pandas` for crunching numbers, once you reach that phase
- Environment: use a virtual environment (`venv`) and a `requirements.txt`. Never commit API keys. Use a `.env` file and add it to `.gitignore`.

Functional requirements, by phase

Each phase produces something that runs. You are never more than one phase away from a working program.

Phase 1: One question, one model

Send a single hardcoded question to one model, get the answer back, print it.

Acceptance criteria: - Runs from the command line - API key loaded from environment, not written in the code - Handles the case where the API call fails without crashing with a raw stack trace

Phase 2: Many questions from a file

Load a list of questions from a file. Loop through them. For each, get the model answer and check it against the correct answer. Print a final score.

Start with multiple-choice questions because grading them is trivial (exact match on the letter). Write 15 to 20 of your own questions in a domain you personally understand well enough to know the right answer cold. Do not use a topic you would have to look up.

Acceptance criteria: - Questions and answers live in a separate file, not in the code - Outputs number correct, number total, and percentage - One bad question or one failed API call does not kill the whole run

Phase 3: Config-driven

Adding a new model or a new question set should require editing a config file, not rewriting code. Separate the harness (the engine) from the data (questions) and the settings (which models, which file, parameters).

Acceptance criteria: - A config file specifies which models to run and which question set to use - Adding a second model is a config change, zero code change - Running two models back to back works and reports both

Phase 4: Make it robust and accountable

Real API calls fail, rate-limit, and cost money. Handle all three.

- Add retry with exponential backoff for failed or rate-limited calls
- Run requests concurrently so a 100-question set does not take 20 minutes. Start simple, do not over-engineer.
- Track tokens used and estimated cost per model per run

Acceptance criteria: - A transient network error is retried, not fatal - Concurrency is noticeably faster than the sequential version, and you can explain the tradeoff - The report shows total tokens and estimated dollar cost per model

Phase 5: Harder scoring

Multiple choice only gets you so far. Add the ability to grade open-ended answers.

- First: fuzzy or normalized matching (case, whitespace, punctuation) for short free-text answers
- Then: LLM-as-judge, where you use a model to grade another model's answer against a rubric

When you build the judge, deliberately test it on answers you know are right and answers you know are wrong. Find a case where the judge gets it wrong. You will. That is the lesson: the judge is itself a fallible model.

Acceptance criteria: - Can grade at least one open-ended question set - You can show me at least one example where the LLM judge graded incorrectly, and explain what happened

Phase 6: Results and reporting

Persist every run with enough detail to reproduce it: timestamp, model, parameters, every question, every answer, every score. Generate a summary report comparing models.

Acceptance criteria: - Past runs are saved and can be re-opened later - A summary report (table is fine, a chart is a bonus) compares models on score and cost - You can answer "what did model X actually answer for question 7 in the run from Tuesday"

Phase 7: Methodology review

No new features. This phase is about understanding what you built well enough to distrust it correctly. Be ready to discuss, with evidence from your own tool:

- Run the exact same eval twice without changing anything. Did the score change? Why?
- What does temperature do to your results?
- Your question set is small. Roughly how much could the score swing by luck alone with that few questions?
- What is data contamination, and why might a model "know" your test answers already?
- If a model vendor publishes "92% on benchmark Y," what would you need to know before believing it means anything?

Acceptance criteria: - A short written section in your README answering the above, using numbers from your own runs

Definition of done

- One command runs multiple models against a question set and produces a saved, comparable report with scores and costs
- Failures are handled gracefully

- The README explains how to set it up, how to run it, and includes the Phase 7 methodology notes
- You can walk me through any file in the project and explain what it does and why

Stretch goals (only after done)

- A second task domain (for example, a small coding-task set graded by running the code)
- A simple web view of results
- Prompt-sensitivity test: same question, three phrasings, measure how much the score moves

Suggested schedule

This assumes a few focused hours per week. It is a guide, not a deadline. The pace is deliberately tight, so getting a phase right matters more than hitting the week. Tell me early if you are stuck, not at the check-in.

Week	Focus	Goal by end of week
1	Setup, Phase 1, Phase 2	Environment and git repo set up, one question to one model working, then a question file with a scoring loop and a percentage on your own multiple-choice set
2	Phase 3	Config-driven, two models running from one config with zero code changes between them
3	Phase 4	Retries and backoff, concurrency, and token/cost tracking all working
4	Phase 5	Fuzzy matching, then LLM-as-judge, with a documented case where the judge graded wrong
5	Phase 6, Phase 7, cleanup	Persistent results and a comparison report, methodology writeup, README, and a code walkthrough with me

Check-ins

Weekly, 20 to 30 minutes. Bring your `NOTES.md`. We review the code and the thinking, not just whether it runs. Do not disappear for two weeks and surface with something big and broken. Small and working beats big and broken every time.